Java Backend Interview — OS & JVM Summary

Q 1 — What Happens When Running `java -jar app.jar`

1. Terminal Layer

- Terminal (bash/zsh) is already a running process.
- It asks the OS to create a new process (`fork + exec` on Linux).

2. OS Layer

The OS: - Loads the `java` ve executable into memory. - Allocates virtual memory space. - Sets heap and stack. - Assigns file descriptors (stdin, stdout, stderr). - Schedules the process for execution.

At this point, a **native OS process** exists.

3. JVM Bootstrap

Inside that process: - The JVM runtime is initialized. - The JVM loads classes from the JAR internal system threads. - It creates the main application thread. - Then it invokes `main()`.

Correct Mental Model

OS Process └── JVM Runtime └── Your Java Application

Key Interview Takeaway

Running `java -jar app.jar`: 1. Creates an OS process. 2. Starts a JVM inside that process 3. Executes Java bytecode within that JVM.

Q 2 — Threads Before `main()` Runs

Important Principle

Before your `main()` method executes, the JVM has already created multiple threads.

Thread Categories

1. OS Bootstrap Thread

• The initial native thread created by the OS.

2. JVM System Threads (Created During Initialization)

Examples (varies by JVM and GC algorithm): - Main thread (created before executing user Garbage Collector threads - JIT compiler threads - Signal handler thread - Reference handler thread

There is **no fixed number** of threads. It depends on: - JVM implementation - GC algorithm - C count - JVM configuration flags

Why This Matters for Backend Engineers

• GC threads affect latency and throughput.
• Thread dumps include many JVM internal threads.
• High thread count does not automatically mean a bug.
• Production debugging requires distinguishing:
• Application threads
• Framework threads
• JVM internal threads

Interview-Grade Statement

"Before `main()` runs, the JVM has already initialized several internal system threads such as JIT compiler threads. The exact number depends on the JVM and configuration."

Q3 — Threads in Production & Thread Dumps

Scenario

• `400` threads in production
• High CPU
• Slow responses

1. Is `400` Threads Automatically Bad?

No. - Depends on server model (Tomcat, Netty, etc.) - Depends on thread states - Depends and pool configuration

What matters more than count: - RUNNABLE vs BLOCKED vs WAITING - Are threads doing useful work?

Interview line: "Thread count alone is not a problem. Thread state and contention determine impact."

---

2. Classifying Threads in a Thread Dump

JVM Internal Threads

Common names: - GC Thread / G1 Conc - VM Thread - Reference Handler - Finalizer Dispatcher - C1/C2 CompilerThread

Framework Threads

Examples: - Tomcat: http-nio- - Netty: nioEventLoopGroup- - *ForkJoinPool.commonPool-worker-* scheduler- */ pool-*

Application Threads

- Created via ExecutorService / @Async / custom pools
- Identified via stack frames in your package namespace

Key Rule: "Thread name gives hints. Stack trace gives truth."

---

3. What to Check First in a Thread Dump

A. Many RUNNABLE Threads

Possible causes: - CPU-heavy loops - Serialization overhead - Logging overhead - Crypto operations

B. Many BLOCKED Threads

Indicates lock contention. Look for: - "waiting to lock" - Which thread owns the monitor

Common causes: - synchronized bottlenecks - Shared resource contention - Logging locks

C. Many WAITING / TIMED_WAITING Threads

Often normal (idle pool threads), but check for: - DB pool exhaustion - HTTP client pool saturation

---

Important Distinction

Thread dump → Debugs thread state & contention. Heap dump → Debugs memory leaks & retention.

Do not confuse thread issues with memory leak issues.

Interview-Grade Takeaway

When diagnosing high CPU + slow responses: 1. Classify thread states. 2. Identify contention saturation. 3. Separate JVM internal threads from application threads. 4. Use heap dumps memory analysis, not thread dumps.

Q3.1 — 200 Threads BLOCKED on Same Lock

Step-by-Step Debugging

1. Identify the Monitor

Find repeated lines: - waiting to lock <0xABC>

2. Find the Lock Owner

Search for: - locked <0xABC>

The thread holding that monitor is the bottleneck.

3. Inspect Lock Owner Stack Trace

Look at the exact source line: - Which class? - Which method? - Is it inside a synchronized block?

Critical Insight

Thread dumps identify contention. The lock owner's stack trace identifies the source code.

Correctness Fix (Most Important)

If the lock owner performs I/O (DB call, HTTP call, file/logging) while holding the lock:

✔️ Move I/O outside the synchronized block. ✔️ Reduce lock scope to protect only in-memo shared state. ✔️ Replace coarse locks with finer-grained or concurrent structures.

Timeouts help reduce blast radius, but they do NOT solve the architectural issue.

Interview line: "Timeouts mitigate impact, but the real fix is removing blocking I/O from sync sections."

Q3.2 — Same Lock Owner Across Multiple Dumps

If 3 dumps taken 10s apart show: - Same thread - Still holding the lock - State = RUNNABLE

This suggests: - CPU-bound work inside synchronized block - Long computation under lock - busy loop

Next Steps: 1. Take multiple dumps to confirm persistence. 2. Use profiler (JFR / async-prof identify hot method. 3. Refactor to reduce work inside critical section.

Senior-Level Takeaway

When diagnosing contention: - Identify monitor. - Identify owner. - Identify why lock is held Fix architectural locking issue, not just symptoms.

Thread Pool Starvation (Spring MVC + Tomcat)

Core Problem

Request threads block while waiting on limited downstream resources (DB, HTTP, async pool). load, all Tomcat threads become busy → no thread left to serve new requests.

Classic Mistake

Using async but still blocking: - @Async + .get() / .join() - WebClient + .block()

This defeats async and causes starvation under load.

Correct Fix Patterns

Fix A — Return Async Type From Controller

Return CompletableFuture (or similar) directly. Tomcat releases request thread while work execute Connection stays open until result is ready.

Important: - Thread is freed - Socket remains open - Client waits normally

Fix B — Keep Fully Synchronous

If you need the result immediately, avoid fake async. Simpler and often safer.

Async MVC Key Insight

Async frees threads, not connections. The browser connection remains open until response completes.

Interview-Level Statement

"In Spring MVC, request threads are the scarce resource. Starvation happens when they block on constrained downstream resources. The fix is to either return async responses properly or flow fully synchronous — but never async and block."